

Data Analytics (DA)

Module -4

Python Assignment 3–While Loop, For Loop, and Function

Assignment Task Overview

This assignment focuses on implementing **loops, control statements, and functions** in Python.

The objective is to develop an understanding of:

- Iteration using a **while loop** and a **for loop**
- Control statements such as **break, continue, pass, and else**
- Function creation and usage
- Logical problem-solving using real-world examples

You will build programs, including a **number guessing game**, a **multiplication table generator**, and a **BMI calculator**.

Tasks:

Task 1: While Loop & Control Statements -Number Guessing Game

Problem Statement: Create a Python program that implements a simple number guessing game using a **while** loop. The program should make use of control statements such as **else, break, and continue**.

Instructions:

1. Set Up the Game:

- ★ Generate a random number between 1 and 10 that the user has to guess. Import random and use randint function.

```
import random  
  
# Generate random number between 1 and 10  
secret_number = random.randint(1, 10)
```

2. Prompt the User:

- ★ Ask the user to guess the number.
- ★ Set a variable **attempts** to 3, which represents the maximum number of guesses allowed.

3. Implement the Guessing Logic:

- ★ Use a **while** loop to allow the user to keep guessing until they get the correct number or run out of attempts.
- ★ Provide feedback to the user for each guess:
 - If the guess is out of the valid range (1 to 10), inform the user.
 - If the guess is greater than the secret number.
 - If the guess is lower than the secret number.
 - If the guess is correct, congratulate the user and end the game.

4. Control Statements:

- ★ Use **continue** to skip the rest of the loop after informing the user if the guess is out of range.
- ★ Use **break** to exit the loop when the guess is correct.
- ★ Use **else** with the **while** loop to provide a message like *"Better luck next time!"* if the user runs out of attempts without guessing the correct number.

Sample Output:

Guess the number (between 1 and 10): 2

Too low. Try again.

Guess the number (between 1 and 10): 15

Your guess is out of range. Please guess a number between 1 and 10.

Guess the number (between 1 and 10): 5

Too high. Try again.

Guess the number (between 1 and 10): 3

Congratulations! You guessed the correct number.

Task 2:- For Loop - Multiplication Table Generator

Problem Statement: Create a Python program that generates and prints a multiplication table (from 1 to 10) for a given number using a **for** loop and the **range** function.

Step-wise Instructions:

1. Prompt user for input. Ask the user to enter a number for which they want to generate a multiplication table.
2. Generate the Multiplication Table: Use a **for** loop to iterate through the numbers 1 to 10. In each iteration, calculate the product of the user's number and the current number from the loop.
3. Display the Multiplication Table:
Print each line of the multiplication table in the format "number x i = result."

Sample Output:

Enter the number for which you want the multiplication table: 5

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50

Task 3: Function- BMI Calculator

Problem Statement: Create a Python program that calculates the Body Mass Index (BMI).

Hint: $\text{BMI} = \text{weight (kg)} / [\text{height (m)}]^2$

Instructions:

1. Define a function `calculate_bmi(weight, height)` that returns the BMI.
2. Prompt the user for their weight (in kg) and height (in meters).
3. Use the function to calculate and display the BMI

Sample Output:

```
Enter your weight in kg: 58
Enter your height in meters: 1.62
Your BMI is: 22.10
```

Deliverables

Submit a project folder containing:

- Complete the assignment in Google Colab or a Jupyter Notebook.
- For Google Colab, share the notebook link with "View" access. If using Jupyter, upload the notebook to Google Drive and ensure "View" access is enabled.
- Put both the Google Colab notebook in a Google Drive folder and share that folder with view access

Skills to be Evaluated and Mapping

Assignment Section	Core Skill	Sub-Skill	Proficiency
While Loop	Control Flow	Iterative execution using while loop	Beginner
Control Statements	Program Control	Using <code>break</code> , <code>continue</code> , <code>else</code>	Intermediate
Number Guessing Logic	Problem Solving	Applying conditions and user interaction	Intermediate
For Loop	Iteration	Using <code>range()</code> for looping	Beginner
Multiplication Logic	Data Processing	Generating sequences and calculations	Beginner
Functions	Code Modularity	Defining and calling functions	Beginner
Function Parameters	Data Handling	Passing arguments (weight, height)	Beginner
Input Handling	I/O Operations	Accepting and validating user input	Beginner
Output Formatting	Data Presentation	Displaying formatted results	Beginner

DA Skill Taxonomy Structure

Skill / Component	Full Marks	Criteria for Full Marks	Partial Credit Description	Partial Marks Range
While Loop & Logic	3	Correct loop usage with attempts and conditions	Minor logical or loop errors	1 – 2
Control Statements	2	Proper use of <code>break</code> , <code>continue</code> , <code>else</code>	Incorrect or missing control statements	1
For Loop Program	2	Correct multiplication table using <code>range()</code>	Minor formatting or logic errors	1
Function Implementation	3	Correct BMI function with accurate calculation	Minor errors in function or formula	1 – 2

DA Skill Taxonomy Structure

- **Looping Concepts:** Use `while` and `for` loops for iteration
- **Control Flow:** Apply `break`, `continue`, `else`, `pass` effectively
- **Problem Solving:** Implement logic for number guessing game
- **Functions:** Define reusable functions with parameters
- **Input/Output Handling:** Accept input and display formatted output

Evaluation Guidelines

Review the submitted **.py script** to ensure:

- The while loop and guessing logic are correctly implemented
- Control statements (**break**, **continue**, and **else**) are used properly
- The multiplication table is generated correctly
- The BMI function works accurately

For each section:

- Map work to defined skills
- Assign marks as per the rubric

Marking Criteria:

- **Full marks:** Correct logic, accurate output, clean code
- **Partial marks:** Minor errors or incomplete implementation

Additional Checks:

- Outputs are correct
- Code is readable and well-commented
- Feedback includes strengths and improvements

Achievement Criteria & Passing Rules

- Minimum marks required in all sections
- Must demonstrate at least two key skills:
 - Looping & Control Statements
 - Functions
- Programs must be executable and logically correct
- Proper input/output handling required
- Provide a **scorecard with feedback** for each submission